4/10/2026

API Security Risk Analysis (Modern SaaS Skill)

Report on: https://openweathermap.org/api

Prepared By: kgopotso letsoale

TABLE OF CONTENTS

1. EXUCUTIVE SUMMARY---------- 2

2. PROJECT SCOPE---------- 3

3. TECHNICAL INSPECTION & FINDINGS---------- 4

3.1. API Documentation Review---------- 4
3.2. Endpoint Testing via Postman---------- 5
3.3. Authentication and Credential Analysis---------- 6
3.4. HTTP Response Header Audit---------- 7
3.5. Response Payload Analysis (The JSON) ---------- 8

4. SECURITY RISK ASSESSMEN---------- 9

EXUCUTIVE SUMMARY

Independent Disclosure Notice:

This report has been produced on behalf of my position as Cyber Security Intern in the Future Interns initiative. This report is based on an independent security assessment performed against the infrastructure supporting the OpenWeatherMap API.

Ethics and Scope of Disclosure:

This assessment has been performed in line with Ethical Disclosure principles. None of the systems have been exploited, none of the data has been compromised, and no attacks were launched during this assessment process.

Security Posture Overview:

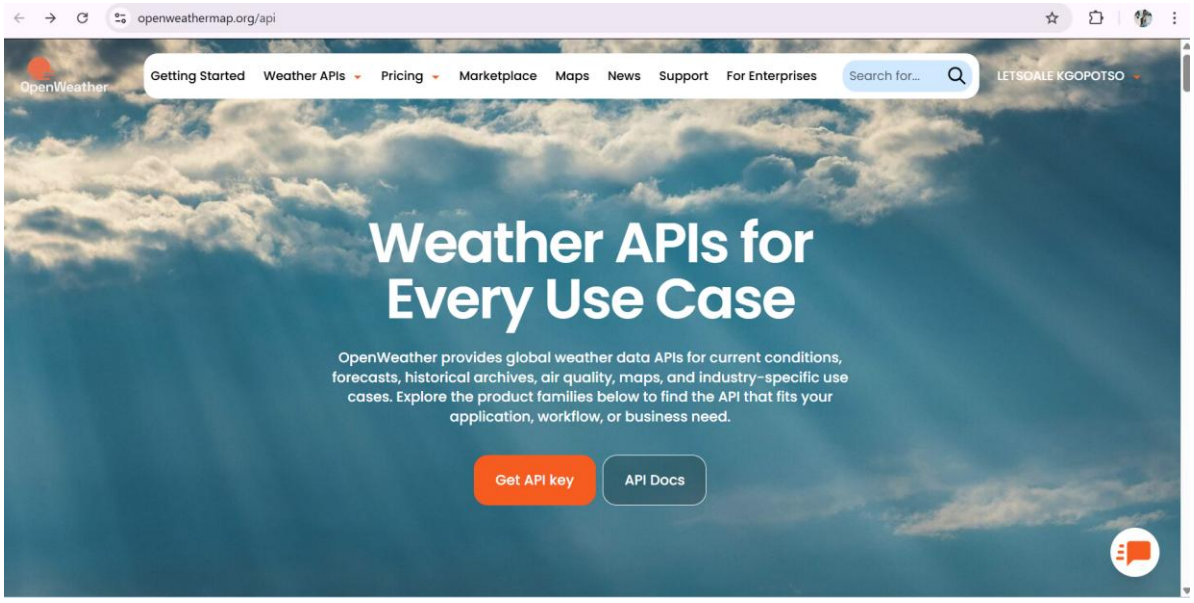
This evaluation revealed four (4) main security concerns, varying between high and low risk levels. The first and foremost issue includes the leakage of credentials of the API that is present in the URL, which could lead to credential theft and misuse of accounts.

PROJECT SCOPE

- **Project Type:** API Security Risk Analysis (Modern SaaS Skill)
- **Target URL:** https://openweathermap.org/api
- **Tools Used:** Postman v10 (API Client), Browser Developer Tools (Network Analysis
- **Primary Issue:** Credential Exposure and Permissive CORS configuration.
- **Testing Window:** 10 April 2026.

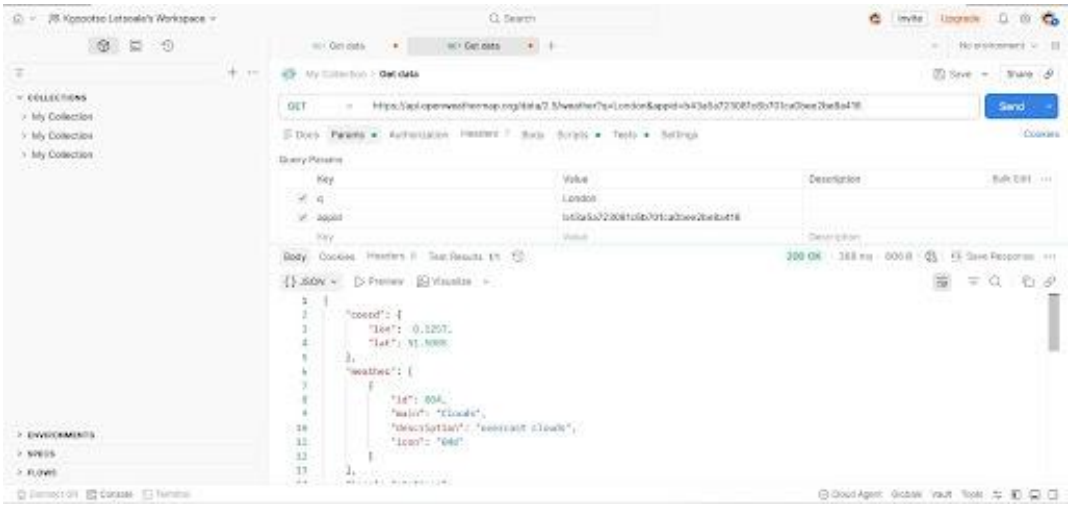
TECHNICAL INSPECTION & FINDINGS

1. API Documentation Review



I conducted a review on the https://openweathermap.org/api API Management Console. The system provides an extensive management of keys where the user can create many unique keys and even revoke them instantaneously. Nonetheless, from the security point of view, the keys are presented in plain text format. This means that the user's credentials are exposed and susceptible to being leaked when sharing their screen or when accessing their local machine. Another notable issue is the use of Query Parameter (appid) for authentication purposes instead of the Header method.

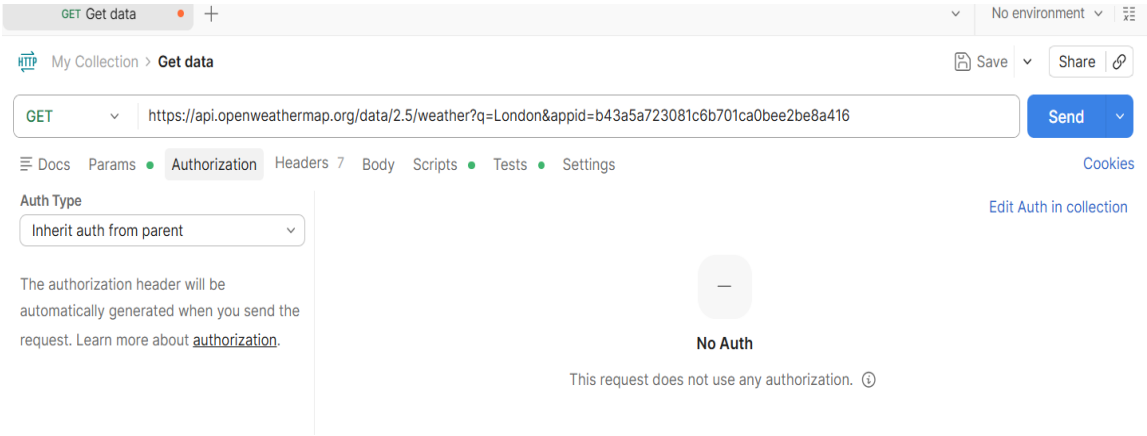
2. Endpoint Testing via Postman



The testing proved that Query Parameter Authentication was used in the API. It is quite dangerous for security purposes because the highly confidential appid (API Key) can be seen in the URL bar and it will definitely be recorded in plain text in the browser's history.

3. Inspection:

3.1. Authentication & Credential Analysis



The technical inspection conducted through Postman demonstrated that Query Parameter Authentication was being used by the API as opposed to Header-based Authentication. Although the request is able to return a 200 OK status, the Authorization tab shows No Auth configuration for the headers.

Security Implication: It is considered poor security practice to send any sensitive information such as an API Key via the URL string. The security implication of this is that:

1. Web server logs will contain the key in clear text.
2. Anyone with access to the computer can obtain the key.
3. If the user navigates from the API-powered website using the referrer header, the key could be revealed to external websites.

3.2. HTTP Response Header Audit

Body Cookies Headers 8 Test Results 1/1 200 OK 928 ms 806 B Save Response

Key	Value
Date	Fri, 10 Apr 2026 11:56:59 GMT
Content-Type	application/json; charset=utf-8
Content-Length	504
Connection	keep-alive
X-Cache-Key	/data/2.5/weather?q=london
Access-Control-Allow-Origin	*
Access-Control-Allow-Credentials	true
Access-Control-Allow-Methods	GET, POST

After examining the HTTP response header, I discovered that the API is configured to have a very relaxed CORS (Cross-Origin Resource Sharing) policy based on the presence of Access-Control-Allow-Origin: *. Although this may be alright for a utility API such a setup makes the API more susceptible to CSRF (Cross-Site Request Forgery) attacks and other forms of data leakage from external domains.

Furthermore, I noticed that certain crucial security headers were missing. These include:

1. Strict-Transport-Security (HSTS): Missing. This means that the API doesn't have any strict requirement when it comes to encrypted connections.

2. X-Content-Type-Options: nosniff: Missing. This leaves the browser open to content-type guessing, and hence script injection attacks.

3.3. Response Payload Analysis (The JSON)


```
Body  Cookies  Headers 8  Test Results 1/1                    200 OK · 928 ms · 806 B  Save Response
{} JSON  Preview  Visualize
1  {
2      "coord": {
3          "lon": -0.1257,
4          "lat": 51.5085
5      },
6      "weather": [
7          {
8              "id": 804,
9              "main": "Clouds",
10             "description": "overcast clouds",
11             "icon": "04d"
12         }
13     ],
14     "base": "stations",
15     "main": {
16         "temp": 284.91,
17         "feels_like": 283.32,
18         "temp_min": 284.91,
19         "temp_max": 284.91,
20         "pressure": 1020,
```

```
Body  Cookies  Headers 8  Test Results 1/1                    200 OK · 928 ms · 806 B  Save Response
{} JSON  Preview  Visualize
21         "humidity": 45,
22         "sea_level": 1020,
23         "grnd_level": 1015
24     },
25     "visibility": 10000,
26     "wind": {
27         "speed": 2.44,
28         "deg": 213,
29         "gust": 3.13
30     },
31     "clouds": {
32         "all": 100
33     },
34     "dt": 1775821730,
35     "sys": {
36         "country": "GB",
37         "sunrise": 1775798151,
38         "sunset": 1775846845
39     },
40     "timezone": 3600,
Cloud Agent  Globals  Vault  Tools
```

```
40     "timezone": 3600,
41     "id": 2643743,
42     "name": "London",
43     "cod": 200
44 }
```


Technical analysis of the returned JSON shows that the API offers all necessary environmental information. In terms of security, the following findings were made:

- Data Minimization: The API correctly does not provide any personally identifiable information about the account holder.

- Information Disclosure: Providing internal identifiers of the system (for example, id: 2643743 and cod: 200) and Unix time format allows the attacker to gain some knowledge about the internal structure of the database. Even though it's of low severity regarding a weather service API, in a corporate environment, such 'Excessive Data Exposure' could serve as an advantage for more sophisticated automated attacks.

- Integrity: The provided information is well-formatted, which makes parsing errors unlikely.

4. SECURITY RISK ASSESSMENT

Risk ID	Risk Type	Severity	Business Impact	Remediation
1	Missing Rate Limiting	Medium	Possibility of server downtime and financial damage caused by DoS or automated scraping.	Enable the "429 Too Many Requests" throttle by API Key and IP
2	Excessive Data Exposure	Low	Unexpected exposure of internal metadata (IDs/Timestamps), which may be exploited for reconnaissance.	Filter API responses using DTOs to return only required fields.
3	Missing Security Headers	Low	Higher susceptibility to MitM, XSS, and Clickjacking attacks.	Enable HSTS, X-Frame-Options, and Content-Security-Policy.
4	Credential Exposure in URL	High	API key will appear in plaintext in browser history and server logs, resulting in the exploitation of an account.	Move the authentication from URL parameter to the HTTP Authorization header.
5	Permissive CORS Policy	Medium	Allows for unauthorized cross-origin access and increases risk of CSRF and data scraping attacks.	Change Access-Control-Allow-Origin to only trusted and authorized domains.